

Rapport sur la flexibilité et les performances

1. Introduction

Ce rapport documente les choix de modélisation, d'indexation et d'agrégation retenus pour le backend du CMS de blog personnel, développé avec Spring Boot, MongoDB et MinIO. L'objectif est d'expliquer pourquoi ces choix favorisent à la fois la flexibilité du modèle de données (propre aux bases documentaires) et les performances des opérations de lecture et d'écriture les plus fréquentes de l'application.

2. Choix de modélisation : référencement vs embarquement

MongoDB permet deux stratégies pour représenter une relation entre deux entités : embarquer un document à l'intérieur d'un autre (dénormalisation), ou le référencer par identifiant dans une collection séparée (normalisation). Le projet combine les deux approches selon la nature de chaque relation.

2.1 Commentaires : référencement (collection séparée)

Les commentaires auraient pu être embarqués directement dans le document **Article**, sous forme de tableau. Ce choix a été écarté au profit d'une collection **Comment** séparée, référençant l'article via un champ **articleId**, pour les raisons suivantes :

- **Limite de taille des documents** : MongoDB impose une limite de 16 Mo par document. Un article très commenté pourrait s'en approcher si les commentaires étaient embarqués, ce qui est un risque non négligeable sur un blog destiné à rester en production durablement.
- **Coût des lectures** : charger un article (par exemple pour l'aperçu dans la liste des articles) n'a pas besoin de charger l'intégralité de ses commentaires. Avec l'embarquement, chaque lecture d'article transporterait aussi tous ses commentaires, même quand ils ne sont pas affichés.
- **Pagination** : afficher les commentaires par page (ex. 20 à la fois) est trivial avec `find().skip().limit()` sur une collection dédiée, alors qu'un tableau embarqué imposerait une extraction manuelle côté application.
- **Écritures concurrentes** : ajouter un commentaire ne modifie que la collection **Comment** ; le document **Article** n'est jamais réécrit pour cette opération, ce qui réduit la contention sur un document potentiellement très consulté.

Le compromis accepté est un coût de lecture légèrement supérieur (deux requêtes au lieu d'une pour afficher un article avec ses commentaires), largement compensé par la flexibilité et la scalabilité obtenues.

Critère	Référencement (Comments, Articles→User)	Embarquement (rejeté ici)
Taille du document	Chaque collection reste légère et bornée	Un article très commenté peut approcher la limite de 16 Mo par document
Lecture d'un article seul	Rapide : le document Article ne charge pas les commentaires	Chaque lecture d'article charge aussi tous ses commentaires, même non affichés
Pagination des commentaires	Simple avec find + skip/limit sur la collection Comment	Nécessite une extraction manuelle depuis un tableau embarqué
Écritures concurrentes	Un nouveau commentaire n'impacte pas le document Article	Chaque nouveau commentaire réécrit tout le document Article
Indexation ciblée	Index dédié sur Comment.articleId	Impossible d'indexer efficacement un tableau imbriqué de la même façon

2.2 Articles vers Auteur : référencement

La relation entre un article et son auteur (**User**) est également gérée par référencement (**Article.authorId**), et non par embarquement des informations de l'auteur dans chaque article. Un utilisateur pouvant rédiger de nombreux articles, dupliquer l'intégralité de ses informations (email, mot de passe hashé, rôle) dans chaque document **Article** introduirait une redondance coûteuse à maintenir et exposerait inutilement des données sensibles. Le référencement via **authorId** garantit une source unique de vérité pour les données utilisateur, et reste indispensable pour la vérification d'autorisation (comparaison avec l'utilisateur connecté lors d'une modification).

Dénormalisation ciblée : **authorName**

Une exception ciblée est faite pour le nom de l'auteur : le champ **Article.authorName** duplique le **fullName** de l'utilisateur au moment de la création de l'article. Cette décision répond à un problème de performance concret : sans cette copie, afficher une liste d'articles avec le nom de leur auteur nécessiterait une requête supplémentaire vers la collection **users** pour chaque article affiché (problème dit « N+1 requêtes »). Avec la dénormalisation, le nom est disponible directement dans le document **Article**, sans jointure applicative.

Le compromis assumé est le suivant : si un auteur modifie son nom après coup, ses articles déjà publiés conservent l'ancien nom affiché, sauf mise à jour manuelle. Ce compromis est jugé acceptable pour ce projet, et suit le même principe que celui déjà retenu pour **Comment.authorName** (voir section 2.1) : favoriser la disponibilité en lecture, caractéristique assumée des bases de données NoSQL orientées documents.

2.3 Images : référencement via URL (ni embarquement, ni GridFS)

Les images associées aux articles ne sont stockées ni en binaire embarqué dans le document MongoDB, ni via GridFS. Elles sont hébergées sur MinIO (compatible S3), et seule l'URL de chaque image est stockée dans le champ **Article.images**. Ce choix découle directement de la limite des 16 Mo par document et des bonnes pratiques de séparation entre stockage de fichiers et stockage de données structurées : MongoDB reste rapide pour interroger les métadonnées des articles, tandis que MinIO est optimisé pour la distribution de fichiers binaires.

3. Pipelines d'agrégation pour les statistiques

L'objectif du projet demandait l'implémentation de pipelines d'agrégation pour produire des statistiques (articles par mois, tags populaires). Ces pipelines sont exécutés via `MongoTemplate.aggregate()`, en écrivant chaque étape sous forme de `Document` MongoDB natif plutôt que via l'API fluide de Spring Data, afin de garder une correspondance directe et lisible avec la syntaxe d'agrégation MongoDB standard.

3.1 Articles publiés par mois

Ce pipeline répond à l'objectif « articles par mois » du cahier des charges. Il se décompose en trois étapes :

```
$project -> extrait "YYYY-MM" depuis Article.createdAt via
           $dateToString
$group   -> regroupe les documents par mois et compte les occurrences
           ($sum: 1)
$sort    -> trie les resultats par ordre chronologique croissant
```

L'étape `$project` transforme un champ de type date en une clé de regroupement textuelle exploitable par `$group`. Sans cette transformation, chaque document aurait un `createdAt` unique à la milliseconde près, rendant tout regroupement par mois impossible directement sur le champ brut.

3.2 Tags les plus populaires

Ce pipeline répond à l'objectif « tags populaires ». Le champ `Article.tags` étant un tableau, l'agrégation utilise l'opérateur `$unwind` pour « déplier » chaque tag en un document temporaire distinct avant de pouvoir les regrouper individuellement :

```
$unwind -> un article possédant 3 tags genere 3 documents temporaires,
           un par tag
$group   -> compte les occurrences de chaque tag distinct
$sort    -> trie par nombre d'occurrences décroissant
$limit   -> conserve les 10 tags les plus frequents
```

Cette approche exploite une caractéristique clé des bases documentaires : la capacité à interroger et transformer des structures imbriquées (tableaux) directement dans le moteur de la base, sans traitement applicatif préalable côté serveur Java.

4. Gestion des index

Les index ont été positionnés sur les champs correspondant aux requêtes les plus fréquentes de l'application, conformément à l'objectif « utiliser des index pour accélérer les recherches par tag ou auteur » du cahier des charges. Sans index, MongoDB effectue un balayage complet de la collection (*collection scan*) pour chaque requête, dont le coût augmente linéairement avec le volume de données.

Collection champ	Type d'index	Justification
User.email	Unique	Empêche deux comptes avec le même email ; accélère <code>findByEmail()</code> utilisé à chaque login

Collection / champ	Type d'index	Justification
<code>Article.authorId</code>	Simple	Accélère <code>findByAuthorId()</code> : lister les articles d'un auteur, utilisé pour le tableau de bord <code>AUTHOR</code>
<code>Article.tags</code>	Multikey (automatique sur tableau)	Accélère <code>findByTagsContaining()</code> : recherche par tag depuis la page publique du blog
<code>Comment.articleId</code>	Simple	Accélère <code>findByArticleId()</code> : chargement des commentaires d'un article, requête la plus fréquente sur cette collection
<code>Article.title</code> , <code>Article.content</code>	Texte (compound text index)	Recherche full-text (<code>GET /api/articles/search</code>) : le titre est pondéré 3 fois plus que le contenu dans le score de pertinence

Le champ `Article.tags` étant un tableau, l'index créé dessus est automatiquement un index multikey : MongoDB indexe chaque élément du tableau individuellement, ce qui permet à `findByTagsContaining()` de retrouver rapidement tous les articles associés à un tag donné, sans balayage de collection.

Contrairement aux index simples ou multikey, MongoDB n'autorise qu'un seul index texte par collection. Les champs `title` et `content` sont donc combinés en un unique index texte, avec une pondération différenciée (`@TextIndexed(weight = 3)` sur `title`) : une correspondance dans le titre contribue davantage au score de pertinence qu'une correspondance dans le contenu. La recherche s'appuie sur l'opérateur `$text` de MongoDB, exécuté via `TextCriteria` et `TextQuery` de Spring Data, et trie les résultats par score décroissant (`sortByScore()`) plutôt que par date de création.

4.1 Garantir la création effective des index

Un point de vigilance a été identifié en cours de développement : depuis Spring Data MongoDB 3.0, la création automatique des index à partir des annotations (`@Indexed`, `@TextIndexed`) est **désactivée par défaut**. Sans configuration supplémentaire, ces annotations documentent une intention de conception sans effet réel en base — aucun index n'est créé, et les requêtes concernées subissent un balayage complet de la collection malgré un code source laissant supposer le contraire.

Pour garantir que les index déclarés existent effectivement, le projet met en place une classe `MongoIndexConfig` implémentant `CommandLineRunner`, exécutée au démarrage de l'application. Elle résout, pour chaque modèle (`User`, `Article`, `Comment`), la liste des index attendus via `MongoPersistentEntityIndexResolver`, puis les crée explicitement via `IndexOperations.createIndex()` s'ils n'existent pas déjà. Cette approche programmatique a été préférée à un simple indicateur de configuration (`auto-index-creation: true`), dont le préfixe de propriété s'est révélé instable d'une version de Spring Boot à l'autre au cours du projet (cf. migration de `spring.data.mongodb.uri` vers `spring.mongodb.uri` en Spring Boot 4) : la création explicite au démarrage constitue une garantie indépendante de ces variations de configuration.

5. Conclusion

Les choix de modélisation présentés dans ce rapport illustrent la flexibilité propre aux bases de données documentaires, qui ne se limite pas à un choix binaire entre embarquement et référencement. Le projet mobilise en réalité trois leviers distincts, combinés selon le besoin de chaque

relation : le référencement pour les relations structurelles où l'intégrité et l'autorisation priment (`Article.authorId`, `Comment.articleId`) ; l'embarquement, volontairement écarté ici pour les commentaires afin d'éviter les risques liés à la taille des documents et à la contention en écriture ; et la dénormalisation ciblée de champs isolés (`authorName`), qui duplique intentionnellement une donnée de lecture fréquente pour éviter des requêtes supplémentaires, sans dupliquer le document entier. Ce dernier levier n'est pas un raccourci technique mais un principe de modélisation à part entière des bases NoSQL orientées documents, où la structure des données suit les patterns d'accès réels de l'application. Les pipelines d'agrégation démontrent par ailleurs la capacité de MongoDB à produire des statistiques directement au niveau de la base, sans traitement intermédiaire coûteux côté application. Enfin, l'indexation ciblée des champs les plus sollicités — y compris l'index texte pour la recherche full-text — garantit que ces choix de flexibilité ne se font pas au détriment des performances ; encore faut-il s'assurer que ces index sont effectivement créés en base, ce que le projet vérifie explicitement au démarrage plutôt que de se fier aux seules annotations. Les objectifs du projet sont ainsi conciliés.